

Clase Object: Raíz de la jerarquía de clases.

Todas las clases extienden directa o indirectamente de la clase Object y por lo tanto una variable de tipo Object puede referirse a cualquier objeto, sea este la instancia de una clase o un array.

Esta clase define algunos metodos que son heredados por todos los objetos. Algunos de estos son: boolean equals, int hashCode, Object clone, entre otras.

El método public boolean equals (Object obj): Compara el objeto receptor con el objeto referido por obj para ver si son iguales.

Si queremos ver si dos REFERENCIAS se REFIEREN AL MISMO OBJETO, usamos == o !=

El método equals pregunta POR LA IGUALDAD DE VALORES.

El método public int hashCode(): HashCode es un identificador de 32 bits cuyo valor depende del estado del objeto o de alguno de sus campos. Este método permite recuperar ese valor.

Si equals indica que dos objetos son iguales, entonces han de tener el mismo valor de hash.

Los métodos hashCode y equals se pueden redefinir si se desea proporcionar una noción de igualdad diferente de la implementación por defecto.

Colecciones: Son receptáculos (no tengo ni puta idea que significa esta palabra) que permiten almacenar y organizar objetos de forma útil para un acceso eficiente.

TIPOS GENERICOS-- PROGRAMACIÓN GENÉRICA

Tipos Genéricos: La programación genérica es un mecanismo para la reutilización de software. El objetivo es independizar el proceso del tipo de datos sobre los que se aplica.

Permite la comprobación de tipos correctos en tiempo de compilación.

Declaración explícita del tipo de datos a utilizar.

No es necesario el casting tras obtener el valor.

El uso de Object como una referencia genérica es potencialmente inseguro y no se puede hacer nada para que el programador no cometa un error equivocando el tipo de objeto esperado. Se pueden declarar clases genéricas, que luego se especialicen

(para un determinado objeto de dato) al momento de usarla.

Una declaración de tipos genéricos puede tener múltiples tipos parametrizados, separados por comas.

Todas las invocaciones de clases genéricas son expresiones de una clase. Al instanciar una clase genérica no se crea una nueva clase.

CONSIDERACIONES PARA USAR DATOS PARAMETRIZADOS:

No se puede usar T (tipo de dato parametrizado) como tipo de un campo estático o de cualquier método dentro de un método o inicializador estático.

No se puede usar un tipo de datos parametrizado en la creación de objetos y arreglos.

No se puede instanciar o crear un array de T por la misma razón que no se puede crear un campo estático de tipo T.

No hay manera de que el compilador sepa el tipo de datos en que va a instanciarse T.

Las colecciones son genéricas.

RESTRICCIONES CON TIPOS GENERICOS

Los parámetros de tipo no pueden ser instanciados. --> No es posible crear una instancia de un parámetro de tipo ya que el compilador no tiene forma de saber qué tipo de objeto crear.

Restricciones en miembros estáticos. --> Ningún miembro estático puede usar un parámetro de tipo declarado por la clase envolvente.

EJEMPLO: NO SE PUEDE HACER `static T objeto`; NI TAMPOCO `static T getOb() return ob`;

Restricciones de arrays genéricos. --> No se puede instanciar un array cuyo tipo de elemento es un parámetro de tipo.

EJEMPLO: NO SE PUEDE HACER `variable = new T[10]`

Tampoco se puede crear un array de referencias genéricas específicas de tipo.

EJEMPLO: `Gen<Integer> gens[]=new Gen<Integer>[10]; //Error.`

Entrada Salida de datos en Java

La entrada/salida en Java está basada en la noción de streams (flujos).

Un flujo es una abstracción de cualquier secuencia de datos yendo desde una fuente (o productor) hacia un destino (o consumidor).

Tanto para la entrada y la salida orientadas a bytes como las orientadas a caracteres la biblioteca `java.io` ofrece dos subconjuntos de clases:

De bajo nivel ofrece funcionalidad básica para la lectura y la escritura de bytes o de caracteres.

Filtrada ofrece alguna funcionalidad añadida para la escritura y lectura de bytes o caracteres.

Entrada filtrada orientada a bytes: Los flujos filtrados (Filter Stream) añaden funcionalidad a un flujo existente, procesando los datos de alguna forma ofreciendo métodos adicionales que permitan acceder a los datos de otros modos.

Serialización

Consiste en un proceso de codificación de un objeto en un medio de almacenamiento con el fin de transmitirlo a través de una conexión en red como una serie de bytes. Esta serie de bytes pueden ser usados para crear un nuevo objeto que es idéntico en todo al original, incluido su estado interno.

La serialización es un mecanismo ampliamente usado para transportar objetos a través de una red, para hacer persistente un objeto en un archivo o base de datos, etc.

Como los objetos mantienen referencias a otros objetos, estos otros objetos también deben ser almacenados y recuperados con el fin de mantener las relaciones originales.

Para que un objeto sea serializable, se debe implementar la interfaz `java.io.Serializable`.

Para que un objeto sea serializable, todas sus variables de instancia han de ser serializables.

Todos los tipos primitivos en Java son serializables por defecto.

Para serializar objetos de una jerarquía solamente la clase base tiene que implementar la interface Serializable.

Los atributos heredados de las clases que no son serializables, al deserializar se iniciarán con los valores por default, no con el valor que fueron guardados.

Serialización binaria (BIN): Se basa en la forma interna de la clase y la reconstrucción de objetos es frágil. No puede haber cambios entre la codificación y decodificación. Para hacer una serialización binaria es necesario que todos los objetos que queremos serializar implementen la interfaz serializable, sino lanza excepción

XML para persistir datos: En vez de almacenar una representación bit a bit de los valores de campo que determinan el estado de un objeto, se almacenan los pasos necesarios para crear el objeto a través de su API pública.

Java ofrece la clase `java.beans.XMLEncoder` para permitir la persistencia de objetos como documentos XML. Esta clase se encarga de convertir el objeto y todos sus datos (incluidos los campos que también son objetos) a un documento XML.

Las clases Java deben: Ofrecer un constructor sin argumentos.

Exponer las propiedades que constituyen el estado del objeto usando metodos `get/set`.

Independencia entre las propiedades.

CONCURRENCIA

Un método sincronizado no puede ser interrumpido ni por sí mismo desde otro thread ni por otro método sincronizado del mismo objeto.

Un monitor (recurso compartido) puede ejecutar de a un método sincronizado por vez.

Si hubiese algún método del recurso compartido (el monitor es el recurso compartido) que no está sincronizado, otro thread podría estar ejecutando de manera concurrente y si podría estar ejecutando ese método no sincronizado.

Bloqueo de un objeto: Si un thread tiene acceso a ejecutar o enviar un mensaje al Monitor, entonces bloquea a los demás, ya que los otros threads que recurren a metodos sincronizados de ese recurso compartido se van bloqueando.

Un método sincronizado puede llamar a otro método sincronizado.

Cuando una clase extendida redefine a un método sincronizado (en la clase base), el nuevo método puede ser sincronizado o no.

Un hijo de un recurso compartido puede dejar de ser recurso compartido.

Los constructores no necesitan ser sincronizados (porque se ejecuta una sola vez en la vida del objeto)

Al competir por un recurso compartido, no hay garantías del orden en el que se ejecutarán los metodos sincronizados. (no sabemos que thread va a ejecutar algún método synchronize)

Los requerimientos de sincronización son parte de la implementación de la clase (no forma parte del diseño, sino de la implementación)

Si el ambiente es concurrente o no es parte de la clase.

Los métodos estáticos también se pueden declarar sincronizados.

Dos hilos no pueden ejecutar metodos estáticos sincronizados de la misma clase al mismo tiempo.

La adquisición del bloqueo en un método estático sincronizado no afecta a los objetos de esa clase.

Lo estático se bloquea con lo estático, y lo dinámico con lo dinámico.

Al dormirse el thread que cae en el `while (!condition) wait();` se libera el bloqueo del recurso compartido y permite que otro thread bloquee.

Todo se ejecuta dentro de un código sincronizado ya que si no fuera así el estado del objeto no sería estable. (Se rompería la zona crítica)

La condición de prueba debe estar siempre en un bucle (ciclo de repetición), ya que si se despierta el thread es porque se liberó el bloqueo, pero no quiere decir que se haya cambiado la condición.

Cuando sale del `wait` sigue en el `while (!condition)` por eso la aclaración de arriba.

Un objeto de una clase que implementa `Runnable` no puede ejecutar `.start()` ya que es propio de la clase `Thread`. Lo que tengo que hacer es: O hacer que la clase se extienda de `Thread` y dejar de implementar `Runnable`, o crear en el `main` un `Thread` para cada objeto `Runnable`, y pasarle por parámetro del constructor del `Thread` el objeto.

Luego de crear los `Threads` a mano, ejecutar el `.start()` con estos hilos.

Si quisiera hacerlo con los objetos `Runnable` debería darle `.run()` a cada uno y esto estaría mal debido a que se ejecutarían de manera secuencial.

Si el recurso compartido estaba bloqueado y este se libera, los que estaban en lista de espera (los threads que quisieron utilizar el recurso, pero estaba bloqueado) toman el control (uno de ellos aleatoriamente)

El `notifyAll` despierta todos los hilos de espera (libera el bloqueo) (ya sea los threads que quisieron utilizar el recurso y estaba bloqueado o los que estaban en el `wait` del ciclo `while`)

Puede haber sentencias `synchronized` dentro de un método que no lo es.

Ventajas de la sentencia synchronized:

La sentencia synchronized permite definir una región del código sincronizada más pequeña que un método.

Permite sincronizar sobre objetos diferentes de this.

Permite un bloqueo selectivo.

Desventajas de la sentencia synchronized:

Se está delegando la responsabilidad del sincronismo al thread y no al recurso compartido como antes.

SEMAFOROS

Un semáforo es un objeto que permite o impide el paso de otros objetos.

Para adquirir un permiso o esperar se utiliza acquire() y para liberar el permiso al haber finalizado la tarea el release()

semáforos parciales: Un semáforo parcial no da garantías sobre quién recibirá un permiso para liberarse.

semáforos imparciales: otorga el permiso según el orden de bloqueo.

Corromper la zona crítica del recurso compartido: Es cuando más de un hilo utiliza el recurso compartido, pisando valores que no debería. En el ejemplo de Lazurri, el recurso compartido era el Pulsador y la zona crítica es el contador. Los hilos la corrompen ya que se están cruzando y rompiendo los tiempos de ejecución.

Este problema de arriba se soluciona haciendo el método clic como synchronized para que el hilo que entra a usar el Pulsador bloquee el recurso compartido.

Los tiempos de espera nunca tienen que estar dentro del recurso compartido.

PATRON OBSERVER

Es un patrón que sirve para establecer una relación entre objetos y se usa cuando se necesita informar a otros objetos el cambio de un objeto.

Se genera una dependencia entre objetos de modo que, cuando un objeto cambia su estado, todos sus dependientes son notificados y actualizados automáticamente.

Este patrón te permite mantener la consistencia entre objetos relacionados, sin aumentar el acoplamiento entre clases.

Se utiliza cuando: Una clase depende de la otra.

Un cambio en un objeto requiere que se actualicen otros sin saber cuántos se van a actualizar.

Un objeto debería ser capaz de notificar a otros objetos sobre su cambio sin necesidad de saber quiénes son esos objetos.

El observador implementa interfaz y el observable hereda de clase abstracta.

El observable (clase que va a ser observada) tiene una colección de observadores. Una vez que en este objeto pasa algo significativo, este emite una notificación a todos los observadores.

El observador mantiene una referencia a cada objeto observado (puede ser más de uno). El método update (que es propio de la interfaz que implementa) se ejecuta cuando los observados notifica. Es responsabilidad del observador mantener vínculo con el observado.

El observable utiliza los métodos setChanged y notifyObservers para notificar los cambios a los observadores.

El update es invocado de manera indirecta, no se llama haciendo ojo.update(). Esto lo genera el notifyObservers.

PATRON STATE (de comportamiento)

Permite que un objeto modifique su comportamiento (un mismo método haga cosas diferentes) cuando su estado interno cambie. Como si el objeto cambiara de clase.

El patrón State es el encargado de encapsular los diferentes métodos que se van a ejecutar en el Context (objeto principal) dependiendo del estado de este.

Los cambios de estado se producen a través de transiciones. Estas transiciones podrían compararse con nuestros métodos, que actúan sobre los valores del objeto haciendo que este cambie o no de estado.

private State estado es el atributo que representa el estado actual de la clase.

Cada método que implementa la interfaz State tiene una referencia al objeto principal para saber el contexto del objeto y poder modificar su estado.

La clase de contexto es la encargada de albergar el estado actual junto al resto de los atributos que se consideran oportunos.

Cada vez que voy a cambiar el estado tengo que usar un setEstado(new nuevoEstado(referencia al objeto principal))

Cada método implementa la transición de metodos. Para esto vuelve a setear el estado del objeto (lo que dije arriba)

Si una clase ya se extiende de Observable y quiero que sea un hilo, es necesario que implemente Runnable ya que Java no permite herencia múltiple.

Si al notifyObservers no le pasas el cambio que hiciste, entonces tienes que obtener el cambio mediante el getter de la clase Observable.

Los observers son completamente independientes de los observadores.

El observador es responsable de generar el vínculo observador observable. Es responsable que genere la consistencia en la doble referencia.

FRAMEWORK

Estructura genérica que se extiende para crear una aplicación o un subsistema más específico.

Apoyan la reutilización de diseño que ofrecen una arquitectura que sirve como esqueleto para la aplicación, así como la reutilización de clases específicas en el sistema.

Se implementan como una colección de clases.

Un framework puede trabajar con otro framework.

Permite inversión de control. El framework me provee el control. Ya viene con un diseño, con un objetivo.

Tiene extensibilidad.

EL PATRÓN MVC

Patrón de arquitectura de software que utiliza tres componentes (Modelo, vista y controlador). Separa la lógica de la aplicación de la lógica de la vista de una aplicación. Es una evolución más en la modularización.

El modelo es manipulado por el controlador y actualiza la vista.

El controlador es usado por el usuario.

La vista es actualizada por el modelo y observada por el usuario.

El modelo: Conjunto de clases que representa la información del mundo real. No puede depender del contexto.

El modelo se separa en dominio y de la aplicación.

Modelo de dominio: Conjunto de clases que se modela al analizar el problema que desea resolver.

Modelo de la aplicación: Conjunto de clases que se relacionan con el modelo del dominio, tiene conocimiento de las vistas y que implementan mecanismos necesarios para notificar cambios en estas vistas.

La vista: Conjunto de clases que se encargan de mostrar al usuario la información contenida en el modelo. Solo obtiene del modelo la información que necesita para desplegar y se actualiza cada vez que el modelo del dominio cambia (por medio de notificaciones generadas por el modelo de la aplicación)

El controlador: Objeto que se encarga de dirigir el flujo del control de la aplicación gracias a mensajes externos (como datos introducidos por el usuario). A partir de este controlador se puede determinar el siguiente paso de ejecución.

Se encarga de modificar el modelo o de abrir y cerrar vistas. Tiene acceso a estas, pero no conocen de la existencia del controlador.

PASOS DE INTERACCIÓN DE LAS COMPONENTES

El usuario interactúa con la interfaz de usuario de alguna forma.

El controlador recibe la notificación de la acción solicitada por el usuario. Gestiona el evento que llega mediante un gestor de eventos. Este accede al modelo y lo actualiza. Delega a los objetos de la vista la tarea de desplegar la interfaz de usuario.

La vista obtiene sus datos del modelo para generar la interfaz apropiada para el usuario donde se reflejan los cambios en el modelo. EL MODELO NO DEBE TENER CONOCIMIENTO DIRECTO SOBRE LA VISTA.

JAVA SWING

Si bien se basa en el patrón de diseño MVC, presenta algunas diferencias de implementación. El controlador y la vista están implementados como un único elemento denominado "delegado de interfaz de usuario" (UI delegate)

Manejo de eventos: Cuando un usuario realiza una acción a nivel de la interfaz de usuario, esto produce un evento. Estos son objetos que describen lo que ha sucedido.

El manejo de eventos de la GUI está basado en el modelo de DELEGACIÓN. Este modelo se basa en objetos que disparan u originan eventos llamados fuentes de eventos y objetos que escuchan y atienden esos eventos llamados escuchas de eventos o listeners.

Objetos fuentes o sources: Las componentes básicas de la GUI (botones, listas, campos de texto) son los objetos que disparan eventos. Estos registran o eliminan los listeners que se interesan en un evento determinado.

GUI en Java: Las GUIs nos permiten ingresar datos.

Las componentes básicas (botones, campos de texto, etc.) son los objetos que disparan eventos. (es lo que está arriba xd)

Sobre una componente AWT se puede registrar una o más escuchas. El orden en que las escuchas son notificadas del evento es indefinido.

Un evento es generado por una componente como consecuencia generada por un usuario (ya escribí esto 10 veces)

Un listener es un objeto que implementa una determinada interface. Los metodos de estos objetos reciben como parámetro un evento específico que contiene información del evento y que objeto AWT lo disparó.

Para cada categoría de eventos hay una interface que debe ser implementada por la clase listener. Cada interface tiene uno o más metodos que deben ser implementados y serán invocados cuando ocurre un

evento específico sobre la componente. (en la diapositiva 33 del pdf Interfaces Graficas hay varias interfaces para aplicar)

Una clase adaptadora es una clase abstracta que implementa la interfaz correspondiente pero no tiene los metodos abstractos, sino que implementa la totalidad de los metodos dejandolos vacíos. De esta forma, se simplifica la construcción de nuestra clase Listener, ya que solo debemos sobrescribir el método que nos interesa.

La desventaja es que extender de una clase adaptadora nos ata a una línea de herencia.